

Q1. Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

Apache Spark follows a Master-Worker architecture to process large datasets in a distributed environment. Every Spark application consists of three important components:

Driver

Cluster Manager

Executors

Each component has a specific responsibility, and together they enable Spark to process data efficiently and in parallel.

User Program



+-----+ +-----+ +-----+

| | |

Task 1 Task 2 Task 3

| | |

|-----|-----|

|

Send Results

|



Driver

1. Driver

The Driver is the main program of a Spark application.

It is responsible for controlling the entire Spark job.

2. Cluster Manager

The Cluster Manager manages the resources of the cluster.

It does not process data.

3. Executor

Executors are the worker processes that perform the actual computation.

They receive tasks from the Driver and execute them in parallel.

Q2: How does Spark's Lazy Evaluation strategy improve performance when processing large datasets?

Spark's Lazy Evaluation means that transformations such as `filter()`, `select()`, and `groupBy()` are not executed immediately. Instead, Spark records these operations and creates a Directed Acyclic Graph (DAG) representing the sequence of transformations. Execution starts only when an action like `show()`, `count()`, `collect()`, or `write()` is called. Before execution, Spark analyzes the DAG and optimizes the entire workflow by combining multiple transformations into fewer execution stages. This reduces unnecessary computations,

minimizes data movement between nodes, and decreases disk I/O. As a result, Spark processes large datasets more efficiently and uses cluster resources effectively. Lazy Evaluation also avoids executing intermediate steps that are not required, improving overall performance. This optimization enables faster execution, lower memory usage, and better scalability when handling big data applications.

```
df = spark.read.csv("sales.csv", header=True, inferSchema=True)

result = df.filter(df.amount > 1000) \

.select("customer_name", "amount")

# No execution happens until this action

result.show()
```

Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

To read a CSV file in PySpark while treating the first row as column headers and automatically detecting the data types, use the `spark.read.csv()` method with the options `header=True` and `inferSchema=True`.

Code:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("ReadCSV").getOrCreate()

df = spark.read.csv(

"data/source.csv",

header=True,

inferSchema=True

)

df.show()
```

Q4: What is the difference between CSV and Parquet in terms of storage (row-based vs. columnar) and why does it matter for performance?

CSV and Parquet are two common file formats used in Apache Spark for storing data. CSV (Comma-Separated Values) is a row-based storage format, where all values of a row are

stored together. It is human-readable and easy to edit but consumes more storage space because it does not support compression or schema storage. Parquet is a columnar storage format, where values of the same column are stored together. It automatically stores the schema, supports efficient compression, and enables Predicate Pushdown, allowing Spark to read only the required columns and rows. As a result, Parquet significantly reduces disk I/O, memory usage, and query execution time. Therefore, Parquet is preferred for big data analytics, while CSV is mainly used for data exchange and importing/exporting datasets.

Q5: Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.

The following PySpark query filters the DataFrame to include only rows where the category is Electronics and selects only the product_id and price columns.

Code:

```
from pyspark.sql.functions import col

result = df.filter(col("category") == "Electronics") \
.select("product_id", "price")

result.show()
```

Explanation:

filter() selects only the rows that satisfy the given condition.

col("category") == "Electronics" filters records belonging to the Electronics category.

select() retrieves only the required columns (product_id and price).

show() displays the filtered results.

Q6: Write the code to "revise" a DataFrame by renaming the column old_name to new_name and casting the price column from a String to a Double.

In PySpark, the withColumnRenamed() function is used to rename a column, while the cast() function is used to convert a column from one data type to another. In this example, the column old_name is renamed to new_name, and the price column is converted from String to Double for performing mathematical operations.

Code:

```
from pyspark.sql.functions import col
```

```
df = df.withColumnRenamed("old_name", "new_name")

df = df.withColumn(

    "price",

    col("price").cast("double")

)

df.printSchema()
```

Explanation:

withColumnRenamed() changes the column name.

cast("double") converts the price column from String to Double.

Converting data types allows Spark to perform calculations such as addition, multiplication, and aggregation.

printSchema() verifies that the changes have been applied successfully.

Q7: How does Spark use the Lineage Graph (DAG) to provide fault tolerance if a worker node fails?

Apache Spark provides fault tolerance using a Lineage Graph, also known as a Directed Acyclic Graph (DAG). Instead of storing multiple copies of data, Spark records every transformation (such as filter(), select(), and groupBy()) performed on the DataFrame. These transformations form a DAG that represents the complete execution plan. If an Executor or worker node fails, Spark does not restart the entire application. Instead, it uses the Lineage Graph to identify the lost partitions and recomputes only the missing data from the original source. This approach saves time, memory, and storage because only the affected partitions are recalculated. As a result, Spark ensures reliable processing of large datasets without maintaining unnecessary replicas, making it highly fault-tolerant and efficient.

DAG Illustration :

Read CSV

|



Filter Data

|



Select Columns

|



Group By

|



Action

(show/write)

If an Executor fails,

Spark recomputes only the missing partition

using the above DAG.

Q8: Write a query to filter a DataFrame df_orders for rows where the status is 'Completed' AND the amount is greater than 1000.

The following PySpark query filters the DataFrame to return only those orders whose status is "Completed" and amount is greater than 1000.

Code :

```
from pyspark.sql.functions import col
```

```
result = df_orders.filter(
```

```
(col("status") == "Completed") &
```

```
(col("amount") > 1000)
```

```
)
```

```
result.show()
```

Explanation :

filter() returns rows that satisfy the specified condition.

& represents the logical AND operator in PySpark.

col() is used to reference DataFrame columns.

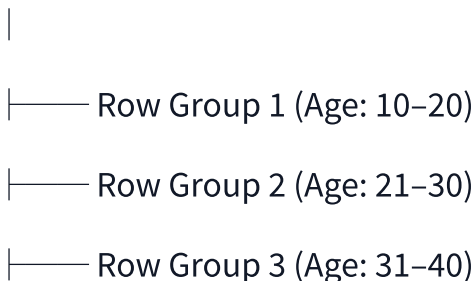
show() displays the filtered records.

Q9: Explain the concept of Predicate Pushdown in Parquet and how it affects the amount of data loaded into memory.

Predicate Pushdown is a performance optimization technique used by Apache Spark when reading Parquet files. Instead of loading the entire dataset into memory, Spark pushes the filtering condition down to the Parquet storage layer. The Parquet file stores metadata such as minimum and maximum values for each column. Spark uses this metadata to identify which row groups satisfy the filter condition and skips the remaining data. As a result, only the required rows and columns are read from disk. This significantly reduces disk I/O, memory consumption, and network traffic. Predicate Pushdown also improves query execution time because Spark processes a much smaller amount of data. Therefore, it is one of the major reasons why Parquet performs much faster than CSV for analytical workloads.

Illustration :

Parquet File



Query:

Age > 30

Spark reads only:

✓ Row Group 3

Spark skips:

✗ Row Group 1

✗ Row Group 2

Q10: Write a code snippet to add a new column `final_price` which is the `base_price` multiplied by 1.18 (18% tax).

The `withColumn()` function is used to create a new column in a DataFrame. Here, the `final_price` column is calculated by multiplying the `base_price` by 1.18, which includes an 18% tax.

Code :

```
from pyspark.sql.functions import col

df = df.withColumn(
    "final_price",
    col("base_price") * 1.18
)

df.show()
```

Explanation :

`withColumn()` creates a new column or updates an existing one.

`col("base_price")` refers to the original price column.

Multiplying by 1.18 adds 18% tax to the base price.

The resulting `final_price` column contains the updated values.

Q11: What is the difference between Transformations and Actions? Provide two examples of each.

In Apache Spark, operations are divided into Transformations and Actions. Transformations are operations that create a new DataFrame or RDD from an existing one without executing immediately. Instead, Spark records these operations and waits until an action is called. This behavior is known as Lazy Evaluation. On the other hand, Actions trigger the execution of all pending transformations and either return a result to the Driver or save the output to storage. Transformations help Spark optimize the execution plan before processing the data, while Actions produce the final output. This separation improves performance and reduces unnecessary computations.

Example Code :

```
from pyspark.sql.functions import col
```


Transformations

```
filtered_df = df.filter(col("price") > 1000)
```

```
selected_df = filtered_df.select("product_id", "price")
```

Actions

```
selected_df.show()
```

```
selected_df.count()
```

Q12: Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where user_id is null, and save the result as a CSV at "path/to/output".

The following PySpark code reads a Parquet file, removes records where user_id is NULL, and writes the cleaned data as a CSV file.

Code :

```
from pyspark.sql.functions import col
```

```
df = spark.read.parquet("path/to/input")
```

```
filtered_df = df.filter(col("user_id").isNotNull())
```

```
filtered_df.write \
```

```
.mode("overwrite") \
```

```
.option("header", True) \
```

```
.csv("path/to/output")
```

Explanation :

spark.read.parquet() reads the Parquet file.

isNotNull() removes rows containing null values in the user_id column.

mode("overwrite") replaces existing output files if they already exist.

option("header", True) writes column names in the CSV file.

csv() saves the filtered DataFrame in CSV format.

Q13: In Spark Architecture, what is the difference between Client Mode and Cluster Mode?

Apache Spark supports two deployment modes: Client Mode and Cluster Mode. In Client Mode, the Driver program runs on the user's local machine, while the Executors run on the cluster. The client machine is responsible for submitting jobs and receiving the results. If the client machine disconnects or crashes, the Spark application may stop. In Cluster Mode, both the Driver and Executors run inside the cluster. The Cluster Manager launches the Driver on one of the worker nodes, making the application independent of the client machine. Cluster Mode provides better fault tolerance, reliability, and resource utilization. Therefore, Client Mode is mainly used for development and testing, whereas Cluster Mode is preferred for production environments.

Architecture :

Client Mode

User PC (Driver)

|



Cluster Manager

|

Executors

Cluster Mode

User PC

|



Cluster Manager

|

Driver

|

Executors

Q14: Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

The following PySpark query filters the DataFrame and returns rows where either the region is "North" or the priority is "High".

Code :

```
from pyspark.sql.functions import col

result = df.filter(

(col("region") == "North") |

(col("priority") == "High")

)

result.show()
```

Explanation:

filter() selects rows based on a condition.

| represents the logical OR operator in PySpark.

col() references DataFrame columns.

show() displays the filtered records.

Q15: When exploring a dataset, why is it safer to use .show(5) instead of .collect() on a multi-terabyte dataset?

When exploring large datasets in Apache Spark, it is much safer to use .show(5) instead of .collect(). The .show(5) function retrieves and displays only the first five rows of the DataFrame, allowing users to inspect the data without consuming significant memory. In contrast, .collect() transfers all rows from every Executor to the Driver program. For multi-terabyte datasets, this can consume a huge amount of memory, leading to OutOfMemory (OOM) errors or even causing the Driver application to crash. Since Spark is designed for distributed processing, moving the entire dataset to a single machine defeats its purpose. Therefore, .show(5) is the recommended method for previewing large datasets because it is faster, memory-efficient, and much safer.

Example :

```
# Safe for large datasets

df.show(5)
```

Avoid on very large datasets

`df.collect()`